



Procédure de génération d'un fichier licence **.lic**

Guide pas-à-pas à destination de l'éditeur WARYA

Produit	WARYA — Caisse tactile
Version document	1.0 — Février 2026
Audience	Éditeur uniquement (confidentiel)
Pré-requis	Python 3.10+, accès au repo yasdynamic/quickpos

■ ■ Ce document contient les clés de signature éditeur. Garde-le hors du repo public, dans un gestionnaire de mots de passe ou un coffre-fort numérique.

Vue d'ensemble

WARYA utilise un système de licences **hors ligne** basé sur deux mécanismes cryptographiques :

1. Signature Ed25519	Chaque .lic est signé par ta clé privée. WARYA embarque la clé publique correspondante et vérifie l'authenticité au lancement, sans contacter Internet.
2. Chiffrement AES-256-GCM	Le contenu signé est chiffré par AES-GCM dérivé de ton sel éditeur (LICENSE_SALT) + une clé symétrique (LICENSE_AES_KEY). Empêche la lecture/altération.
3. Empreinte machine	Chaque licence est liée à une empreinte hashée du PC client (hostname + adresse MAC + machine-id + sel). Une licence ne fonctionne que sur la machine cible.

Le client ne te connecte jamais à un serveur de licences : il t'envoie son empreinte, tu lui retournes son .lic personnalisé. C'est tout.

1 Le client t'envoie son empreinte

Quand un client lance WARYA pour la première fois (ou après réinstallation sur un nouveau poste), il voit l'écran « **Activer WARYA** » avec une empreinte unique du type :

```
POS-2FB6-4EED-5FDA
```

Il te l'envoie par **WhatsApp, SMS, e-mail ou téléphone**. Récupère également :

- le **nom commercial** du commerce (ex. « Café Matin Bamako »)
- le **plan acheté** : Standard, Pro ou Entreprise
- la **durée** de la licence (1 an, 2 ans, perpétuelle, etc.)
- le **nombre de postes** et d'utilisateurs souhaités

2 Setup initial du poste éditeur (1 seule fois)

Sur ton poste personnel (Linux, macOS ou Windows avec Python ≥ 3.10) :

```
# Cloner le repo
git clone https://github.com/yasdynamic/quickpos.git
cd quickpos/backend

# Installer les dépendances minimales
pip install cryptography python-dotenv
```

Crée ensuite un fichier .env.editor hors du repo Git (jamais commité !) :

```
# .env.editor (à garder secret)
LICENSE_SALT=quickpos-AFR-2026-do-not-share-or-the-fingerprint-changes
LICENSE_AES_KEY=quickpos-aes-secret-v1-DO-NOT-CHANGE-after-activation
```

■ ■ Ces deux valeurs doivent être **identiques** à celles compilées dans WARYA. Si tu les modifies, toutes les anciennes licences cessent de fonctionner. Sauvegarde-les dans un gestionnaire de mots de passe (1Password, Bitwarden, KeePass) et garde un backup chiffré sur clé USB.

3 Générer la licence avec le CLI

Charge les variables d'environnement puis lance `license_gen.py` :

```
# Linux / macOS
export $(cat .env.editor | xargs)

python3 tools/license_gen.py \
--license-number "WARYA-2026-001" \
--company "Yas Dynamic" \
--client-name "Café Matin Bamako" \
--fingerprint "POS-2FB6-4EED-5FDA" \
--edition "Pro" \
--max-users 20 \
--max-workstations 5 \
--years 2 \
--out cafe-matin-bamako.lic
```

✓ Le fichier produit fait environ **400-500 octets**. Il est binaire (magic header QPLIC), ne tente pas de l'éditer. Tu peux l'envoyer en toute sécurité par tout canal — il est inutile pour quiconque ne possède pas physiquement le poste avec l'empreinte cible.

Référence complète des options CLI

Argument	Obligatoire	Description	Exemple
<code>--license-number</code>	Oui	N° unique facture/client	WARYA-2026-001
<code>--client-name</code>	Oui	Nom commercial du client	"Café Matin Bamako"
<code>--fingerprint</code>	Oui	Empreinte envoyée par le client	POS-2FB6-4EED-5FDA
<code>--out</code>	Oui	Chemin du fichier .lic à créer	client.lic
<code>--company</code>	Recommandé	Ton entreprise éditrice	"Yas Dynamic"
<code>--edition</code>	Recommandé	Standard / Pro / Enterprise	Pro
<code>--max-users</code>	Recommandé	Nb max d'utilisateurs	20
<code>--max-workstations</code>	Recommandé	Nb max de postes	5
<code>--years</code>	Durée	Validité en années	1, 2, 5
<code>--days</code>	Durée	Validité en jours	30 (essai)
<code>--perpetual</code>	Durée	Licence à vie (flag)	(sans valeur)

Note : un seul flag de durée parmi `--years`, `--days` ou `--perpetual` doit être utilisé par licence.

4 Envoyer le fichier .lic au client

Par e-mail, WhatsApp, message ou clé USB. Le fichier est petit et personnalisé — il ne fonctionnera que sur le poste possédant l'empreinte exacte.

✓ Tu peux envoyer le .lic en clair (e-mail, messagerie non chiffrée, etc.) sans risque : même intercepté, il est cryptographiquement inutilisable sur une autre machine.

5 Le client active WARYA

Le client lance WARYA, arrive sur l'écran « **Activer WARYA** », puis :

- Vérifie que l'empreinte affichée correspond bien à celle envoyée
- Glisse-dépose le fichier .lic dans la zone d'import (ou clic pour parcourir)
- WARYA vérifie la signature Ed25519 et déchiffre le contenu en local
- Si tout est valide : redirection automatique vers l'écran de connexion

Aucune connexion Internet n'est requise à l'activation. La licence est stockée dans la base locale et vérifiée à chaque démarrage.

Exemples de scénarios courants

Licence d'essai 30 jours

```
python3 tools/license_gen.py \  
--license-number "TRIAL-2026-042" \  
--client-name "Restaurant Test" \  
--fingerprint "POS-XXXX-YYYY-ZZZZ" \  
--edition "Standard" --max-users 3 --max-workstations 1 \  
--days 30 \  
--out trial-restaurant-test.lic
```

Licence perpétuelle Enterprise multi-postes

```
python3 tools/license_gen.py \  
--license-number "WARYA-ENT-2026-007" \  
--client-name "Chaîne Hôtels Sahel" \  
--fingerprint "POS-AAAA-BBBB-CCCC" \  
--edition "Enterprise" --max-users 100 --max-workstations 50 \  
--perpetual \  
--out hotels-sahel-perpetual.lic
```

Licence Pro 1 an, mono-poste

```
python3 tools/license_gen.py \  
--license-number "WARYA-PRO-2026-013" \  
--client-name "Boutique Dakar" \  
--fingerprint "POS-1234-5678-9ABC" \  
--edition "Pro" --max-users 5 --max-workstations 1 \  
--years 1 \  
--out boutique-dakar.lic
```

Astuce : script wrapper pour aller plus vite

Crée generate-license.sh sur ton poste éditeur pour automatiser la commande :

```
#!/bin/bash
# Usage : ./generate-license.sh <fingerprint> <client-name> <edition> <years>
export $(cat /path/to/quickpos/backend/.env.editor | xargs)
cd /path/to/quickpos/backend

NUM="WARYA-$(date +%Y)-$(printf '%03d' $((RANDOM % 1000)))"
python3 tools/license_gen.py \
--license-number "$NUM" \
--client-name "$2" \
--fingerprint "$1" \
--edition "${3:-Pro}" \
--max-users 20 --max-workstations 5 \
--years "${4:-1}" \
--out "./licenses/${NUM}-${(echo "$2" | tr ' ' '-')} .lic"
```

Puis :

```
chmod +x generate-license.sh
./generate-license.sh POS-2FB6-4EED-5FDA "Café Matin Bamako" Pro 2
```

Sécurité opérationnelle

- ✓ Garde .env.editor hors de tout repo Git, même privé.
- ✓ Stocke LICENSE_SALT + LICENSE_AES_KEY dans un gestionnaire de mots de passe.
- ✓ Backup chiffré (clé USB chiffrée, coffre cloud) à un autre emplacement physique.
- Ne discute jamais ces clés en clair (e-mail, Slack, Discord, etc.).
- Tiens un registre Excel/Notion des licences émises (n°, client, empreinte, expiration).
- ✗ **Si elles fuient** : tu dois changer les clés, refaire signer toutes les licences, et recompiler WARYA. Catastrophique.

FAQ rapide

Q. Le client change de PC, que faire ?

R. Demande la nouvelle empreinte de la nouvelle machine, génère un .lic avec la même --license-number mais la nouvelle empreinte. Tiens un registre pour traçabilité.

Q. Le client réinstalle Windows, l'empreinte change ?

R. Possible si la carte réseau / machine-id change. Génère une nouvelle licence avec la nouvelle empreinte (gratuit côté éditeur). Vérifie qu'il s'agit du même client.

Q. Comment révoquer une licence existante ?

R. Pas de révocation en ligne (système 100 % offline). Pratique : génère une nouvelle licence expirée immédiatement (--days 0) avec la même --license-number et demande au client de l'importer. Solution future : système de listes noires CRL.

Q. Plusieurs licences pour un même client multi-postes ?

R. Génère un .lic par poste (chaque PC a son empreinte). Garde le même --license-number ou utilise des suffixes (WARYA-2026-001-A, -B, etc.).

Q. Que contient exactement le .lic ?

R. JSON signé (Ed25519) puis chiffré (AES-256-GCM) : nom du client, fingerprint, dates, plan, quotas, numéro éditeur. Aucune donnée personnelle sensible.

— Fin de la procédure —